Lab 4: Profiling Naive vs FFT Convolution (imgconv_fft)

GitHub Classroom link

In this lab you will learn profiling by investigating a real C++ program that applies convolution filters to an image. The program supports two convolution methods:

- **naive** (direct spatial convolution)
- **fft** (FFT-based convolution)

Your goal is not to "make the code faster."
Your goal is to answer, with evidence:

- Where does the runtime go?
- Why does performance differ between naive and FFT?
- When is FFT slower, and when can it win?

What profiling is (and what it is not)

Profiling answers: **where does the runtime go?** It helps you find *hot spots* (functions/loops that dominate total time), and it helps you verify that your optimization efforts are aimed at the right place.

- Profiling is **not debugging**.
- Profiling is **not guessing**.
- We measure, change one thing, measure again.

Two useful ideas:

- **Wall-clock timing**: "How long did the whole thing take?"
- **Sampling profilers**: "Which functions/lines are we spending time in?"

In this lab, chrono timing is already built into the program. Your job is to use:

- `perf` (Linux)
- Instruments Time Profiler (macOS)

Important: FFT is NOT always faster

Do not assume "FFT beats naive."

- For **small kernels** (3×3 sharpen/edge/emboss), naive typically wins because FFT has large overhead.
- FFT becomes competitive only when the kernel is **large enough** (large blur), and implementation constants are reasonable.

Your job is to **measure** and then explain your results.

Part 0 — Setup and sanity check

Build

```HTML
make clean
make
```

Use the canonical input image

All commands below assume the input image is:

```HTML
data/oldkenyon.ppm
```

Sanity run (confirm program works)

```HTML
./imgconv --in data/oldkenyon.ppm --filter emboss --method naive --repeat 5 --ou
./imgconv --in data/oldkenyon.ppm --filter emboss --method fft   --repeat 5 --ou
```

If both images look reasonable, you are ready to measure.

Part 1 — Timing experiments (chrono built-in)

Run each command at least twice. Use the average reported by the program.

Experiment 1: A small-kernel filter (FFT should lose)

```
HTML
./imgconv --in data/oldkenyon.ppm --filter emboss --method naive --repeat 10 --
./imgconv --in data/oldkenyon.ppm --filter emboss --method fft   --repeat 10 --
```

Experiment 2: Larger blur (FFT might win)

```
HTML
./imgconv --in data/oldkenyon.ppm --filter blur --kernel-size 51 --method naive
./imgconv --in data/oldkenyon.ppm --filter blur --kernel-size 51 --method fft
```

Experiment 3: Find the "crossover" (if any)

Try blur kernel sizes:

- 3, 7, 15, 31, 51, 101

Record runtime for naive and FFT at each size.

Part 2 — Profiling option A: Linux `perf` (sampling profiler)

Perf answers: **which functions dominate CPU time**.

Build with symbols

```
HTML
make clean
make CXXFLAGS="-O3 -g -march=native -Wall -Wextra -pedantic"
```

Profile FFT blur (choose a workload long enough to sample)

Increase repeat until the run lasts ~1–3 seconds.

```
HTML
perf record -g ./imgconv --in data/oldkenyon.ppm --filter blur --kernel-size 51
perf report
```

Profile naive blur (same workload idea)

```
HTML
perf record -g ./imgconv --in data/oldkenyon.ppm --filter blur --kernel-size 51
perf report
```

How to interpret `perf report`

Focus on:

- **Top 1–3 entries** (the hotspots)
- **Self vs Children**
- Call graph expansion (you used `-g`)

Write down:

- the top hotspot function names
- the approximate percent
- what phase they correspond to (FFT phase, multiply phase, naive inner loop, etc.)

Part 3 — Profiling option B: macOS Instruments Time Profiler

Please refer to what we did in class to getr MAc profiling set up with xcode. Link
Instruments answers: **where CPU time is spent**, with a clear call tree and timeline.

Build with symbols

```
HTML
make clean
make CXXFLAGS="-O3 -g -Wall -Wextra -pedantic"
```

Run Instruments (Time Profiler)

- Open Instruments
- Choose **Time Profiler**
- Select target: `imgconv`
- Program arguments:

```
HTML
--in data/oldkenyon.ppm --filter blur --kernel-size 51 --method fft --repeat 20
```

Record, let it finish, stop.

Interpret results

In Call Tree:

- enable **Hide System Libraries**
- enable **Invert Call Tree**
- find top 1–3 hottest functions

Repeat for naive blur (lower repeat count is fine if it runs long enough):

```
HTML
--in data/oldkenyon.ppm --filter blur --kernel-size 51 --method naive --repeat 5
```

What you submit

1) Results table

Include a table with:

- filter
- kernel size
- method
- average runtime

2) Profiler evidence

Include either:

- a short list of top hotspots + percent, OR
- a screenshot of the call tree/report (optional, but helpful)

3) Explanation (the important part)

In 1–2 pages, answer:

- Why does FFT lose badly for emboss/edge/sharpen?
- For blur, does FFT become competitive? At what kernel sizes?
- For each method, where is the CPU time going?
- Is the performance limited by compute (math) or memory movement?

Grading (quick rubric)

- Correct execution + clean results table: **30%**
- Meaningful profiling evidence: **30%**
- Clear explanation tied to evidence: **40%**

Admin